
1. Recommended Tech Stack (Best Fit)

Core Architecture

React (Frontend)



FastAPI (Backend / Orchestration Layer)



MATLAB Engine API (Processing Core)

2. Why This Stack Wins

Frontend — React (Keep it)

- Strong ecosystem for dashboards
- Excellent charting libraries (Recharts, Chart.js)
- Component-based → easy modular UI

Verdict: Correct choice. No change needed.

Backend — FastAPI (Python)

This is the **critical decision layer**.

Why FastAPI over Node.js:

- Native integration with **MATLAB Engine for Python**
- Handles numerical data (arrays, signals) efficiently
- Async support → faster API responses
- Built-in validation (Pydantic)

👉 Node.js + MATLAB = friction

👉 Python + MATLAB = native synergy

Verdict:

FastAPI is the most efficient orchestration layer for your system.

Processing Layer — MATLAB

- Runs:
 - LMS filter
 - Noise classification
 - Compensation engine
 - Matching logic

Integration Method:

Use:

MATLAB Engine API for Python

Example flow:

```
import matlab.engine
eng = matlab.engine.start_matlab()
result = eng.adaptive_ir_filter_v2(signal)
```

3. Supporting Components (Essential)

Data Exchange Format — JSON

- Lightweight
 - Easy React integration
 - Supports structured outputs (matches, parameters)
-

File Handling

- Upload CSV → FastAPI

- Convert → NumPy array → MATLAB
-

Charting (Frontend)

Use:

- **Recharts** (clean, simple)
OR
 - **Plotly.js** (if you want advanced spectral plots)
-

4. Optional but High-Value Additions

A. WebSocket (Real-time updates)

If you want:

- Live LMS convergence graph
- Step-size (μ) evolution

Use:

- FastAPI WebSockets
-

B. Database (For IR Reference Data)

Option 1 (Simple):

- JSON / CSV (local)

Option 2 (Better):

- SQLite

Option 3 (Scalable):

- PostgreSQL

👉 Start simple. Upgrade later.

5. Tech Stack Summary Table

Layer	Technology	Reason
Frontend	React.js	Best for dashboards
Backend	FastAPI (Python)	Native MATLAB integration
Processing	MATLAB	Core DSP logic
Integration	MATLAB Engine API	Direct execution
Data Format	JSON	Clean API communication
Charts	Recharts / Plotly	Visualization
Storage	JSON / SQLite	IR database

6. Architecture Decision (Blunt Reality)

❌ Avoid:

- Node.js backend (poor MATLAB integration)
- Direct MATLAB → frontend (no control layer)
- Monolithic MATLAB UI (not scalable)

✅ Do:

- Use Python as **control plane**
 - Keep MATLAB as **compute engine**
 - Keep React as **presentation layer**
-

7. Execution Workflow

1. User uploads CSV (React)
 2. FastAPI receives file
 3. Converts → NumPy array
 4. Sends to MATLAB Engine
 5. MATLAB processes:
 - Filtering
 - Classification
 - Matching
 6. Returns structured output
 7. FastAPI → JSON response
 8. React renders dashboard
-

8. Risk Considerations

MATLAB Engine Bottleneck

- MATLAB is not inherently scalable
- Each request may spawn/hold session

Mitigation:

- Maintain a **persistent MATLAB session**
 - Queue requests if needed
-

Latency

- MATLAB execution can be slow for large N

Mitigation:

- Preprocess in Python where possible
 - Optimize MATLAB code (vectorization)
-

9. Final Recommendation

React + FastAPI + MATLAB Engine is the optimal stack.

This gives you:

- Clean separation of concerns
 - Minimal integration complexity
 - Strong academic + industry credibility
-